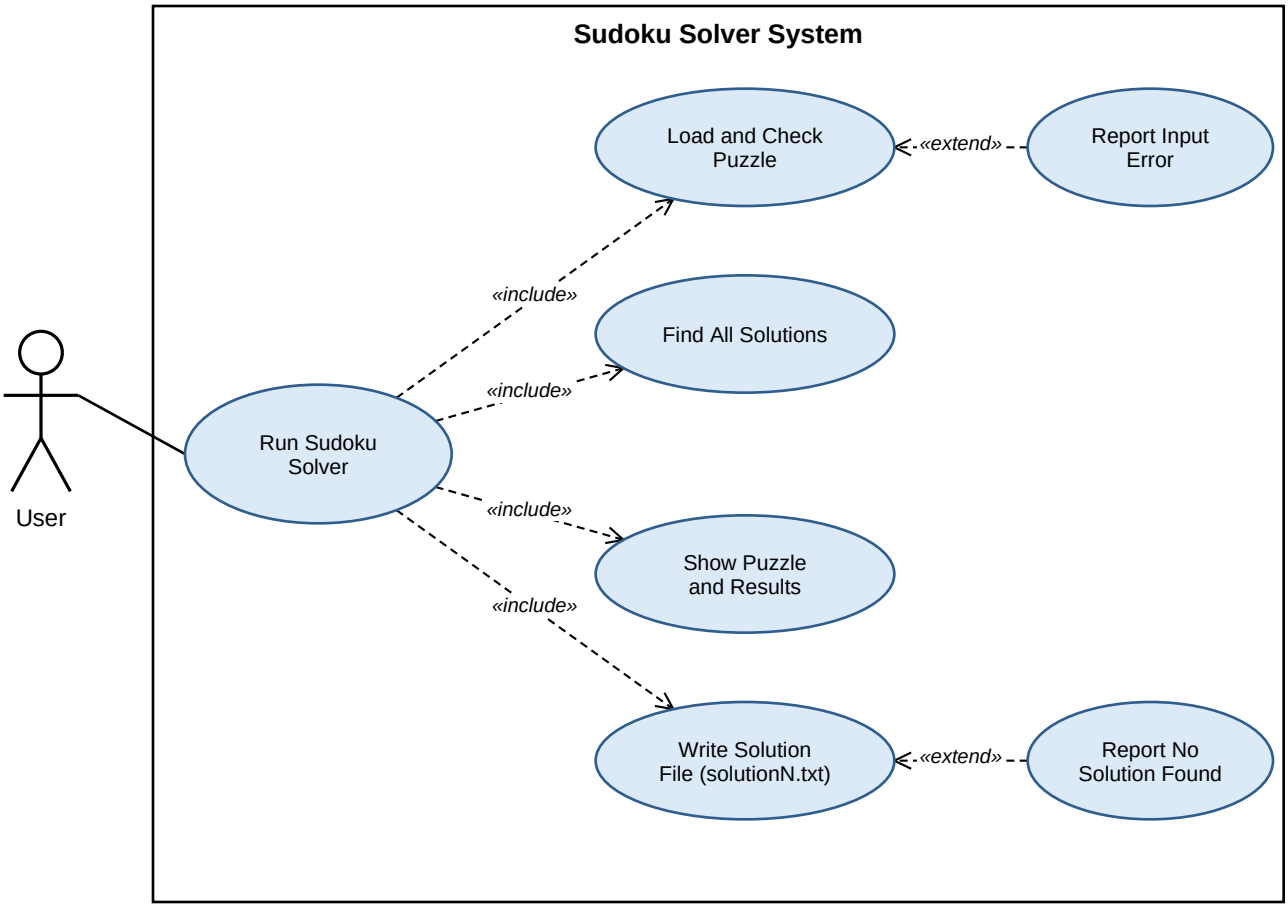
UML Use Case Model: Sudoku Solver

Assignment 5 — EECS 348 Software Engineering I

Original model: Assignment 4, Saylor Statler, October 14, 2026
Revised by: Jagannath Sritinath Nair, October 29, 2026

Use Case Diagram

The diagram shows how the user interacts with the Sudoku Solver System after the Assignment 5 revision. The main use case, *Run Sudoku Solver*, includes loading and checking each puzzle, finding all solutions, showing the puzzle and results, and, new in Assignment 5, writing the results to a solution file (solution1.txt through solution5.txt). *Report Input Error* extends *Load and Check Puzzle* and only happens when a puzzle file is unreadable or invalid. *Report No Solution Found* extends *Write Solution File* and only happens when the search finds nothing.



Identify / Discover the Use Case

Primary actor	User (the person running the program).
System	Sudoku Solver System, a C++ program (Assignment5.cpp).
Use case	Run Sudoku Solver.
Goal	Read each of the five Sudoku puzzle files, find every valid solution, show the results on the console, and save them in the matching solution file.
Preconditions	Assignment5.exe is run from a folder containing puzzle1.txt through puzzle5.txt. The program has permission to create files in that folder.

Postconditions	solution1.txt through solution5.txt exist. Each one holds either all solutions of its puzzle followed by "Solutions found: n", or the single line "No solution found."

Brief Description

The user runs the program. The program opens the five puzzle files (puzzle1.txt through puzzle5.txt), checks that each puzzle is readable and follows the Sudoku rules, and searches for all valid completed boards. It prints each original puzzle and every solution on the console. For each puzzle it also creates a solution file (solution1.txt through solution5.txt) that lists all solutions in the same 9x9 format as the input files, followed by the number of solutions found, or the single line "No solution found." when there is none.

Step-by-Step Flow of Events

Main flow

1. The user starts the Sudoku solver executable.
2. The program processes the five puzzle files in order and pairs each puzzle*N*.txt with solution*N*.txt.
3. For the current puzzle, the program creates (or overwrites) the solution file, then opens the puzzle file and reads the 9x9 grid.
4. The program checks the input format: all 81 cells present, only the digits 1-9 or underscores, and no extra characters.
5. The program displays the original puzzle and checks that the given numbers do not already break a row, column, or 3x3 block rule.
6. If the puzzle is good, the program searches for solutions with recursive backtracking. It always picks the empty cell with the fewest legal digits, tries each legal digit, and undoes any choice that leads to a dead end.
7. Whenever a complete board is found, the program writes it to the solution file in the input-file format (nine rows of nine space-separated characters), displays it on the console, and keeps searching for more solutions.
8. When the search ends with at least one solution, the program writes the final line "Solutions found: n" to the solution file and displays the total on the console.
9. The program closes the solution file and repeats steps 3-8 for the remaining puzzle files, then finishes.

Alternative flows

A1 — No solution (at step 8): The search finishes with zero solutions. The program writes the single line "No solution found." to the solution file, reports the same on the console, and continues with step 9.

A2 — File cannot be opened or format is invalid (at steps 3-4): The program reports the input problem (cannot open, incomplete puzzle, invalid character, or extra characters), skips solving, writes "No solution found." to the solution file, and continues with step 9.

A3 — Conflicting given numbers (at step 5): The program reports that the puzzle is invalid, writes "No solution found." to the solution file, and continues with step 9.

A4 — Solution file cannot be created (at step 3): The program reports that it cannot create the file, skips that puzzle, and continues with step 9.

Detailed Description

When the user runs the Sudoku solver, the system goes through the five puzzle files one at a time. For each puzzle number, it opens the matching solution file for writing, then reads the puzzle file and checks the input: the grid must contain exactly 81 cells, and each cell must be a digit from 1 to 9 or an underscore for an empty cell. Anything else in the file causes the input error to be reported. The system then displays the original puzzle

and checks that the numbers already given do not conflict with each other.

If the puzzle passes these checks, the system uses recursive backtracking to fill the empty cells. At each step it chooses the empty cell with the fewest possible digits, tries each of them in turn, and removes a digit again when it leads to a dead end. This continues until every possible placement has been tried, so every valid completed board is found and not just the first one. Each completed board is written to the solution file immediately, in the same format as the puzzle files, and is also displayed on the console.

After the search is finished, the system ends the solution file in one of two ways. If at least one solution was found, the last line is "Solutions found: n" with n equal to the number of solutions. If none was found, or the puzzle file could not be used, the file contains only the line "No solution found." and the console shows the matching message. Once the file is closed, the system moves on to the next puzzle and stops after all five have been processed, leaving five solution files for the user.